

GAME PROGRAMMING COURSE — C# BASICS

列挙体・クラス・継承

Paiza 演習で身につける C# の基礎（構造体入門の続き）

対象：1年生（構造体まで既習）

ENUM /
CLASS

LESSON 1

列挙体（Enum）

目標：決まった選択肢を持つデータを、列挙体で表せるようになる



このコマの目標

- `int` や `string` で状態を管理することの問題点を体感する
- 列挙体（`enum`）が何のためにあるかを理解する
- 列挙体を定義し、`switch` 文と組み合わせて使えるようになる
- Paiza の問題を一緒に解き、実際に手を動かす

列挙体（enum）とは

- 決まった選択肢の中から1つを表す、名前付きの値をまとめた型
- `public enum 型名 { 値1, 値2, ... }` という形で定義する

```
c# Program.cs
1 public enum PlayerState
2 {
3     Idle,
4     Walking,
5     Jumping,
6     Attacking
7 }
```

POINT

Idle ・ Walking ・ Jumping ・ Attacking は「名前」。0/1/2のような数字を意識しなくてよい。

列挙体を使ってみる

c# Program.cs

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 public enum PlayerState
5 {
6     Idle,
7     Walking,
8     Jumping
9 }
10 class Program
11 {
12     static void Main()
13     {
14         PlayerState state = PlayerState.Idle;
15         Console.WriteLine(state);
16         state = PlayerState.Walking;
17         if (state == PlayerState.Walking)
18         {
19             Console.WriteLine("歩いている");
20         }
21     }
22 }
```

switch 文との組み合わせ

- if を連ねるより、switch を使うと選択肢ごとの処理が見やすくなる。

C# Program.cs

```
1  switch (state)
2  {
3      case PlayerState.Idle:
4          Console.WriteLine(" 待機中 ");
5          break;
6      case PlayerState.Walking:
7          Console.WriteLine(" 歩いている ");
8          break;
9      case PlayerState.Jumping:
10         Console.WriteLine(" ジャンプ中 ");
11         break;
12 }
```

このコマのポイント

- 列挙体は「決まった選択肢」に名前をつけて表す型
- enum 型名 { 値 1, 値 2, ... } で定義する
- 型名 . 値名 で参照する（例： PlayerState.Walking ）
- switch 文と組み合わせると、選択肢ごとの処理が読みやすくなる

LESSON 2

クラス

目標：データと処理をまとめるクラスの基本を理解する



このコマの目標

- 構造体だけでは足りない場面を理解する
- クラス（class）が何のためにあるかを理解する
- クラスを定義し、インスタンス化して使えるようになる
- public と static の意味を理解する
- 構造体とクラスの違いを理解する

前回までの振り返り

- 構造体＝複数のデータをまとめる型
- `public struct PlayerData { public string Name; public int Hp; }`
- 列挙体＝決まった選択肢に名前をつけて表す型
- `public enum PlayerState { Idle, Walking, Jumping }`

LESSON 2

構造体の軽い復習

- 本題に入る前に、前回学んだ構造体の書き方をコードでおさらいしておきましょう。

c# Program.cs

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 public struct PlayerData
5 {
6     public string Name;
7     public int Hp;
8 }
9 class Program
10 {
11     static void Main()
12     {
13         PlayerData p1 = new PlayerData();
14         p1.Name = " ゆうた ";
15         p1.Hp = 100;
16         Console.WriteLine(p1.Name + " " + p1.Hp);
17     }
18 }
```

構造体だけでは、困ることがある

C# Program.cs

```
1 public struct PlayerData
2 {
3     public string Name;
4     public int Hp;
5 }
6 // HP を減らす処理をどこに書く?
7 static void TakeDamage(ref PlayerData p, int dmg)
8 {
9     p.Hp -= dmg;
10 }
```

- 構造体は「データ」しか持てない（フィールドのみ）
- 「HP を減らす」処理は、構造体の外に別の関数として書くしかない
- データと、それを操作する処理がバラバラになってしまう

オブジェクト指向の考え方

- データと処理を、現実の「モノ」のようにひとまとまりにして考えるプログラムの設計スタイル
- 例えば「プレイヤー」は、名前や HP というデータと、ダメージを受けるという処理をあわせ持つ1つの「モノ」
- この「モノ」のことをオブジェクトと呼び、オブジェクトの設計図にあたるのがクラス
 - ー カプセル化・継承・ポリモーフィズムという3つの考え方が土台になる（継承・ポリモーフィズムは LESSON3 で扱う）

POINT

構造体の限界（データと処理がバラバラ）を解消するのが、まさにこの考え方です。

クラス（class）とは

- データ（フィールド）と処理（メソッド）を1つにまとめる型
- 書き方は構造体とほぼ同じで、struct を class に変えるだけ

c# Program.cs

```
1 public class PlayerData
2 {
3     public string Name;
4     public int Hp;
5
6     public void TakeDamage(int dmg)
7     {
8         Hp -= dmg;
9         Console.WriteLine(Name + " はダメージを受けた ");
10    }
11 }
```

クラスを使ってみる

c# Program.cs

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 public class PlayerData
5 {
6     public string Name;
7     public int Hp;
8     public void TakeDamage(int dmg)
9     {
10         Hp -= dmg;
11     }
12 }
13 class Program
14 {
15     static void Main()
16     {
17         PlayerData p1 = new PlayerData();
18         p1.Name = "ゆうた";
19         p1.Hp = 100;
20         p1.TakeDamage(20);
21         Console.WriteLine(p1.Hp);
22     }
23 }
```

public とは

- public は「どこからでもアクセスできる」ことを示すアクセス修飾子
- フィールドやメソッドの前につけることで、クラスの外から使えるようになる
- 反対に private をつけると、そのクラスの中からしかアクセスできなくなる
 - － p1.Name や p1.TakeDamage(20) のように外から呼べているのは、public だから

static とは

- static は「インスタンスを作らなくても使える」ことを示すキーワード
- static がついたメンバーは、クラス全体で 1 つだけ共有される
- 呼び出すときは 変数名 . ではなく、クラス名 . を使う

c# Program.cs

```
1 public class Counter
2 {
3     public static int Count = 0;
4
5     public Counter()
6     {
7         Count++;
8     }
9 }
10 // 呼び出す側
11 Console.WriteLine(Counter.Count);
```

構造体とクラス：ここが違う

struct（構造体）

- データ（フィールド）のまとめ役が中心
- コピーすると「実体ごと」複製される（値型）
- 座標など、小さく単純なデータに向く

class（クラス）

- データ＋処理（メソッド）をまとめられる
- コピーすると「同じ実体への参照」を共有する（参照型）
- キャラクターなど、機能を持つものの管理に向く

POINT

値型・参照型の違いは、今後さらに詳しく扱います。今日は「まとめ方」の違いに注目しましょう。

このコマのポイント

- クラスは「データ+処理」をまとめる型
- `public class 型名 { フィールドとメソッド }` で定義する
- `new 型名 ();` でインスタンス化し、メソッドを呼び出して使う
- `public` : どこからでもアクセスできる / `static` : インスタンスなしで使える
- 構造体は値型、クラスは参照型という違いがある

演習へ： Paiza の問題を一緒に解く

- ここからはクラスの書き方を使って、実際に Paiza の問題を解いていきます。



静的メンバ C# 編

https://paiza.jp/works/mondai/class_primer/class_primer__static_member/edit?language_uid=c-sharp



RPG C# 編

https://paiza.jp/works/mondai/class_primer/class_primer__heros/edit?language_uid=c-sharp



格闘ゲーム C# 編

https://paiza.jp/works/mondai/class_primer/class_primer__fighting_game?language_uid=c-sharp



出口のない迷路 C# 編

https://paiza.jp/works/mondai/class_primer/class_primer__closed_maze/edit?language_uid=c-sharp

LESSON 3

継承・発展

目標：クラスの継承を理解し、共通処理をまとめられるようになる



このコマの目標

- 似たクラスを何度も書く非効率さを体感する
- 継承（クラス名：親クラス名）の書き方を理解する
- メソッドのオーバーライド（virtual / override）を理解する
- ポリモーフィズムの考え方に軽く触れる

導入：似たクラスを何個も書く

C# Program.cs

```
1 public class Player
2 {
3     public string Name;
4     public int Hp;
5     public void TakeDamage(int dmg) { Hp -= dmg; }
6 }
7 public class Enemy
8 {
9     public string Name;
10    public int Hp;
11    public void TakeDamage(int dmg) { Hp -= dmg; }
12 }
```

注意

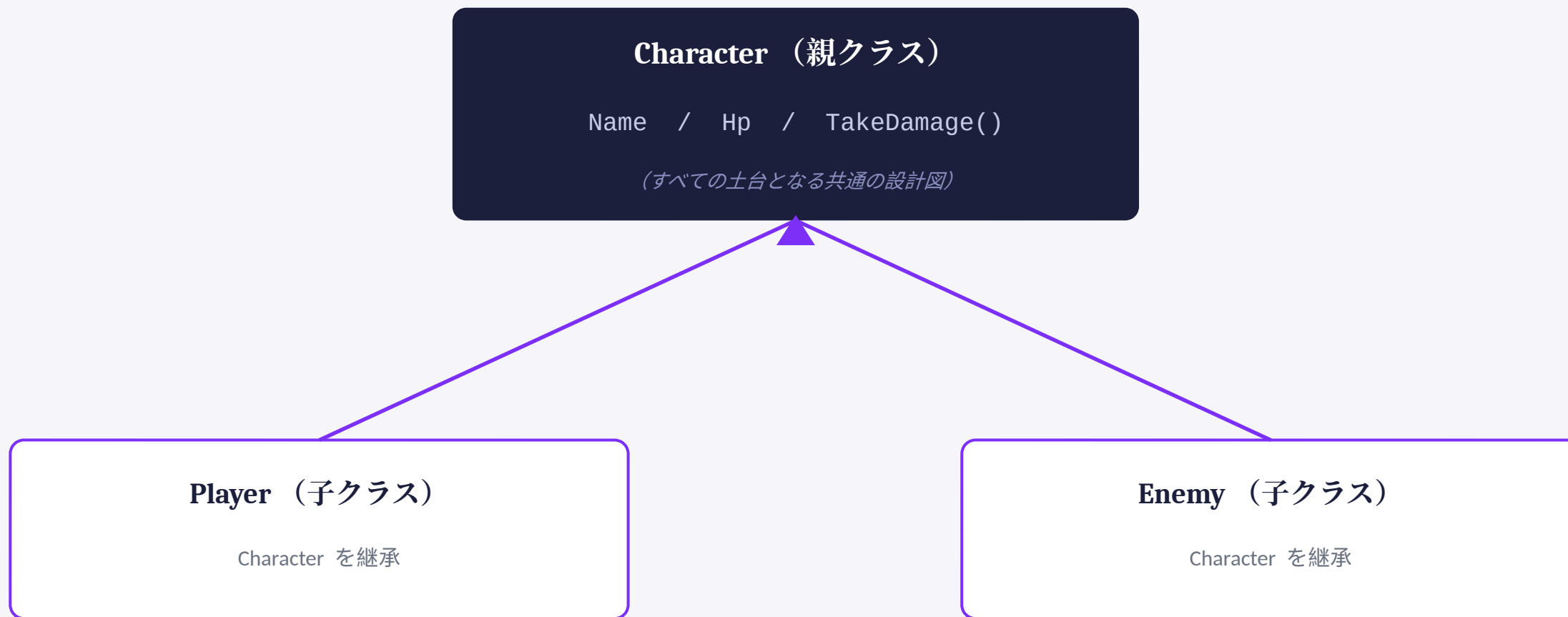
Player と Enemy で、まったく同じフィールド・メソッドを2回書いてしまっている。

継承とは

- 既存のクラス（親）が持つフィールドやメソッドを、新しいクラス（子）がそのまま引き継ぐ仕組み
- `class 子クラス名 : 親クラス名` という形で書く

```
c# Program.cs
1 public class Character
2 {
3     public string Name;
4     public int Hp;
5     public void TakeDamage(int dmg) { Hp -= dmg; }
6 }
7 public class Player : Character
8 {
9 }
10 public class Enemy : Character
11 {
12 }
```


継承の図解



矢印は「継承している」方向（子→親）を表す。Player と Enemy は、Character のフィールド・メソッドをそのまま引き継ぐ。

継承したクラスを使ってみる

c# Program.cs

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 public class Character
5 {
6     public string Name;
7     public int Hp;
8     public void TakeDamage(int dmg) { Hp -= dmg; }
9 }
10 public class Player : Character { }
11 class Program
12 {
13     static void Main()
14     {
15         Player p1 = new Player();
16         p1.Name = "ゆうた";
17         p1.Hp = 100;
18         p1.TakeDamage(20);
19         Console.WriteLine(p1.Hp);
20     }
21 }
```

メソッドのオーバーライド

- 親クラスのメソッドを `virtual` にしておくと、子クラスで `override` して中身を書き換えられる。

c# Program.cs

```
1 public class Character
2 {
3     public string Name;
4     public virtual void Greet()
5     {
6         Console.WriteLine(Name + " です ");
7     }
8 }
9 public class Enemy : Character
10 {
11     public override void Greet()
12     {
13         Console.WriteLine(Name + " が現れた! ");
14     }
15 }
```

発展：ポリモーフィズムに触れる

- 同じ Character 型として扱っても、実際の型ごとに違う Greet が呼ばれる。

C# Program.cs

```
1 Character[] characters = new Character[2];
2 characters[0] = new Player();
3 characters[1] = new Enemy();
4
5 foreach (Character c in characters)
6 {
7     c.Greet();
8 }
```

POINT

呼び出す側は Player か Enemy かを気にせず、c.Greet() と書くだけでよい。これがポリモーフィズム。

このコマのポイント

- 継承＝親クラスのフィールド・メソッドを子クラスが引き継ぐ仕組み
- `class 子 : 親` という形で書く
- `virtual / override` で、子クラスごとに処理を書き換えられる
- 同じ型として扱っても、実際の型に応じて動きが変わる（ポリモーフィズム）

演習へ： Paiza の問題を一緒に解く

- ここからは継承の書き方を使って、実際に Paiza の問題を解いていきます。

POINT

※ このページに演習問題の URL を追加してください。

お疲れさまでした

構造体・列挙体・クラス・継承——ここまでの土台が、
これから作るゲームのしくみを支えていきます。